

Chapter 6

Data Engineering and Automation

Overview of data engineering in applied data science

Data Engineering is the discipline of **designing, building, and maintaining systems that collect, store, process, and deliver data** so that data scientists and analysts can actually use it.

In short:

Data Engineers make data reliable, scalable, and usable.

Data Scientists make data insightful and predictive.

Why Data Engineering is Critical in Applied Data Science

In textbooks, data is clean and ready.

In real life, data is:

- Incomplete
- Messy
- Distributed across systems
- Continuously changing

Applied data science deals with:

- Real users
- Real-time systems
- Production constraints

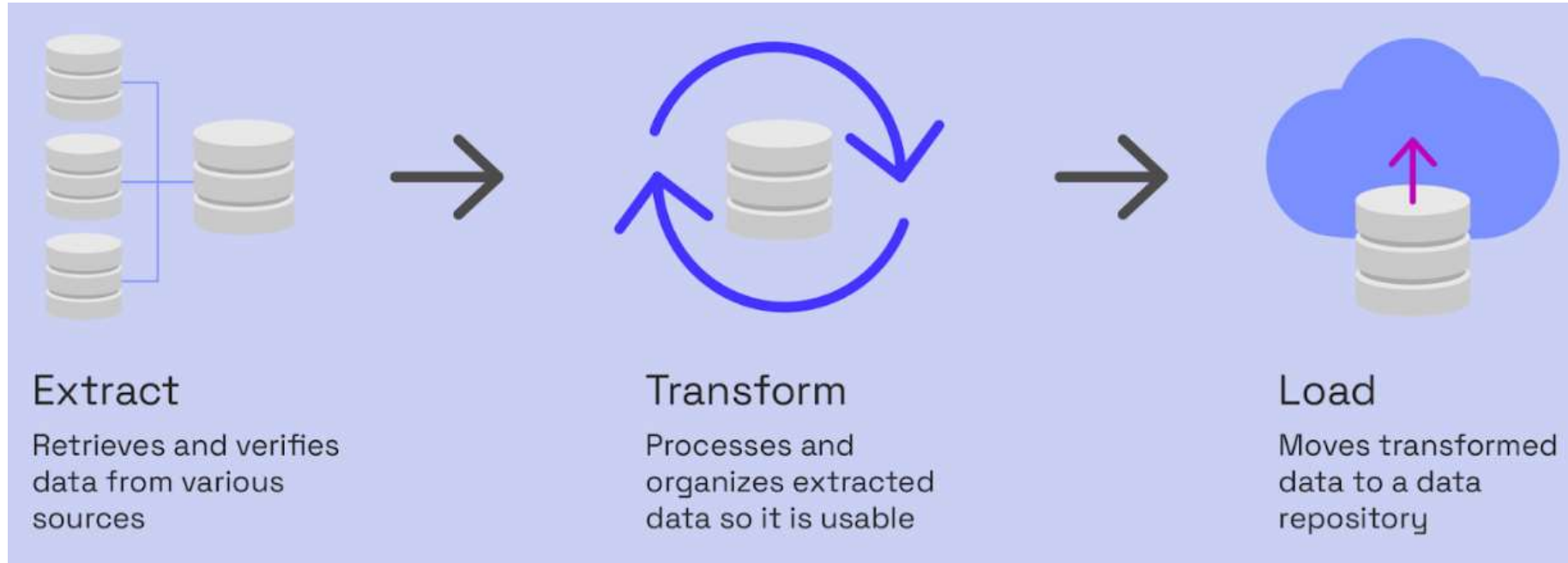
Data engineering ensures:

- Models get **correct and fresh data**
- Pipelines **don't break**
- Systems **scale with users**

Data Science Vs Data Engineering

Aspect	Data Engineering	Data Science
Focus	Data pipelines & infrastructure	Modeling & insights
Data state	Raw → Processed	Processed → Insights
Output	Tables, features, pipelines	Models, predictions
Skills	SQL, ETL, cloud, systems	Statistics, ML, analysis

Designing and implementing ETL pipelines



Designing and implementing ETL pipelines

An **ETL pipeline** is an automated workflow that:

1. **Extracts** data from multiple sources
2. **Transforms** it into clean, consistent, usable form
3. **Loads** it into a target system (warehouse / analytics store)

In applied data science, ETL pipelines turn **raw data into model-ready data**.

ETL Vs ELT

Aspect	ETL	ELT
Transform location	Before loading	After loading
Storage	Smaller systems	Cloud warehouses
Flexibility	Lower	Higher
Modern usage	Declining	Dominant

Designing and implementing ETL pipelines

STEP 1 Identify the Business Objective

Ask: *Why is this data needed?*

Examples:

- Reporting
- Machine learning
- Monitoring
- Personalization

📌 This step decides **everything else** (data, frequency, schema).

STEP 2 Identify Data Sources

Decide:

- Where does data come from?
 - Databases
 - APIs
 - Files
 - Logs
 - Streaming sources

Define for each source:

- Data format
- Volume
- Update frequency
- Ownership

👉 Output of this step: **List of data sources**

Designing and implementing ETL pipelines

STEP 3 Decide Extraction Strategy

Choose how to pull data

Strategy	When to Use
Full extract	Small datasets
Incremental	Large tables
CDC	Real-time systems

📌 Extraction must not overload source systems.

🔥 Output: Extraction logic

STEP 4 Design Staging Area (Raw Layer)

Purpose: Store raw data safely.

Rules:

- No transformation
- Append-only
- Time-partitioned

Why?

- Recovery from failures
- Auditing
- Reprocessing

🔥 Output: Raw data storage design

Designing and implementing ETL pipelines

STEP 5 Define Target Schema

Before transforming anything, decide:

- Table structure
- Column types
- Primary keys
- Relationships

📌 Never transform without knowing the target schema.

👉 Output: **Final table design**

STEP 6 Design Transformation Logic

Now decide how raw data becomes usable data.

Transformations include:

- Cleaning (nulls, duplicates)
- Type conversion
- Standardization
- Aggregation
- Feature creation

Rules:

- Modular
- Reusable
- Idempotent

👉 Output: **Transformation rules**

Designing and implementing ETL pipelines

STEP 7 Decide Loading Strategy

How data is loaded into target

Method	Use Case
Full load	Small tables
Incremental load	Large data
Upsert	Slowly changing data

Also decide:

- Partitioning
- Indexing
- Load order

👉 Output: Load logic

STEP 8 Add Data Quality Checks

Decide what can go wrong and catch it.

Checks:

- Schema validation
- Null thresholds
- Range checks
- Row count comparison

📌 Quality checks are mandatory in real systems.

👉 Output: **Validation rules**

Designing and implementing ETL pipelines

STEP 10 Monitoring, Logging & Failure Handling

Final safety layer.

Decide:

- What to log
- What triggers alerts
- How failures recover

Metrics:

- Runtime
- Record count
- Data freshness



Output: **Operational plan**

ETL pipelines summary

Business Need

→ Data Sources

→ Extraction

→ Staging

→ Schema Design

→ Transformation

→ Loading

→ Quality Checks

→ Orchestration

→ Monitoring

AUTOMATING WORKFLOWS WITH SCHEDULERS

What Does “Automating a Workflow” Mean?

A **workflow** is a sequence of tasks, for example:

Fetch data → Clean data → Save result → Log status

Automation means:

These tasks run **automatically**, at a **fixed time or interval**, without a human clicking “run”.

AUTOMATING WORKFLOWS WITH SCHEDULERS

Why Do We Need Schedulers?

Imagine an ETL script:

`etl.py`

If you run it manually:

- You may forget
- Timing may change
- Data becomes outdated

A **scheduler** solves this by answering:

? WHEN should this script run?

AUTOMATING WORKFLOWS WITH SCHEDULERS

What is a Scheduler (THEORY)?

A **scheduler** is a system that:

1. Keeps track of time
2. Checks scheduling rules
3. Executes a task when time matches

Note:

- Scheduler **does NOT process data**
- Scheduler **ONLY triggers execution**

Types of scheduler

Types of scheduler

Type	Example
System-level scheduler	CRON
Application-level scheduler	Python <code>schedule</code>

What is CRON ?

CRON is a **Linux/Unix system scheduler**.

It runs:

- Scripts
- Commands
- Programs

At:

- Fixed times
- Fixed intervals

CRON runs even if:

- ✓ You log out
- ✓ Server restarts

CRON Architecture

CRON Daemon (runs in background)



Reads crontab



Checks system clock



Executes command

CRON Time format

* * * * *

| | | | |

| | | | └ Day of week (0–6, Sunday=0)

| | | └ Month (1–12)

| | └ Day of month (1–31)

| └ Hour (0–23)

└ Minute (0–59)

Special symbols:

- * → every
- */n → every n units

CRON examples

Task	CRON Expression
Every minute	* * * * *
Every day at 1 AM	0 1 * * *
Every 10 minutes	*/10 * * * *
Every Monday at 9 AM	0 9 * * 1
First day of month	0 0 1 * *

Writing CRON Task

Step 1: Create a script

```
python

# etl.py
print("ETL job running")
```

Step 2: Open cron table

```
bash

crontab -e
```

Step 3: Add job

```
bash

0 1 * * * /usr/bin/python3 /home/user/etl.py >> /home/user/etl.log 2>&1
```

Step 4: Save & exit

✓ Job runs daily at 1 AM

Python “schedule”

What is Python **schedule**?

schedule is a **Python library** for scheduling jobs **inside a Python program**.

Key idea:

The Python process must keep running.

How **schedule** Works

Python script starts



Defines jobs



Loop checks time



Runs job when due

Installing schedule

```
pip install schedule
```

Basic `schedule` Code

```
import schedule

import time

def job():

    print("Job running")

schedule.every(10).seconds.do(job)

while True:

    schedule.run_pending()

    time.sleep(1)
```

Common Scheduling Patterns (CODE)

Every day at fixed time

```
schedule.every().day.at("01:00").do(job)
```

Every hour

```
schedule.every().hour.do(job)
```

Every Monday

```
schedule.every().monday.at("09:00").do(job)
```

Every 5 minutes

```
schedule.every(5).minutes.do(job)
```

Real ETL Example Using **schedule**

```
import schedule

import time

def etl():
    print("Extract data")
    print("Transform data")
    print("Load data")
    print("ETL completed")

schedule.every().day.at("01:00").do(etl)

while True:
    schedule.run_pending()
    time.sleep(60)
```

CRON Vs Schedule

Feature	CRON	schedule
Level	OS	Python
Needs running script	✗	✓
Reliability	High	Low
Best use	Production	Learning
Monitoring	Manual	Manual

Logging, Monitoring & Error Handling in Data Pipelines

A **data pipeline** is useless if:

- you don't know **what happened**
- you don't know **when it broke**
- you don't know **why it broke**

Concept	Purpose
Logging	Record <i>what happened</i>
Monitoring	Observe <i>system health & performance</i>
Error handling	Decide <i>what to do when something fails</i>

LOGGING IN PIPELINES

Logging means **systematically recording events** that happen during pipeline execution.

Examples:

- Pipeline started
- File received
- 1,250 rows loaded
- API request failed
- Transformation completed

Why logging is critical

Without logs:

- Debugging = guessing
- Failures = silent
- Audits = impossible

With logs:

- You know **where** it failed
- You know **why**
- You can **replay or fix**

Logging levels

Level	Meaning	Example
DEBUG	Detailed internal info	Variable values
INFO	Normal execution	"Job started"
WARNING	Something odd	Missing optional column
ERROR	Failed operation	DB connection failed
CRITICAL	System crash	Pipeline aborted

Python logging

```
import logging

logging.basicConfig(
    filename="pipeline.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

def extract():
    logging.info("Extraction started")
    data = [1, 2, 3]
    logging.info(f"Extracted {len(data)} records")
    return data

def transform(data):
    logging.info("Transformation started")
    return [x * 2 for x in data]
```

```
def load(data):
    logging.info(f>Loading {len(data)} records")
    logging.info("Load completed")

try:
    data = extract()
    data = transform(data)
    load(data)
    logging.info("Pipeline completed successfully")

except Exception as e:
    logging.error("Pipeline failed", exc_info=True)
```

MONITORING IN PIPELINES

What is monitoring?

Monitoring means **continuously tracking pipeline health & performance.**

Logging tells *what happened*

Monitoring tells *is everything okay right now?*

MONITORING IN PIPELINES

What do we monitor?

Pipeline-level metrics

- Job success / failure
- Execution time
- Schedule delay
- Retry count

Data-level metrics

- Row count
- Null values
- Duplicate records
- Schema changes

System-level metrics

- CPU
- Memory
- Disk
- Network

MONITORING IN PIPELINES Simple example

```
import time

start = time.time()

# pipeline runs here

duration = time.time() - start

if duration > 300:
    logging.warning("Pipeline took longer than expected")
```

What is error handling?

Error handling defines **how your pipeline reacts to failures**.

Failures are **guaranteed** in real systems.

Common pipeline failures

- Network timeout
- API rate limit
- Corrupt file
- Schema change
- Disk full
- Permission denied

Error handling example

```
import time

def extract_with_retry(retries=3):
    for attempt in range(retries):
        try:
            logging.info("Trying extraction")
            # simulate failure
            raise ConnectionError("API failed")
        except Exception as e:
            logging.warning(f"Attempt {attempt+1} failed")
            time.sleep(2)

    raise RuntimeError("All retries failed")
```

Data Storage and Retrieval Strategies for Data Pipelines

A **data pipeline** moves data from **sources** → **processing** → **storage** → **consumption**.

Storage and retrieval strategies decide **where data lives**, **in what format**, and **how fast & safely it can be accessed**.

Data Storage and Retrieval Strategies for Data Pipelines

Storage Architectures: Where it Lives

The "where" depends entirely on how structured your data is and how fast you need to query it.

- **Data Lakes (Object Storage):** Best for raw, unstructured, or semi-structured data (S3, GCS, Azure Blob). It's cheap and infinitely scalable but can become a "data swamp" without a catalog.
- **Data Warehouses:** Best for structured, relational data (Snowflake, BigQuery, Redshift). These use **columnar storage**, which is optimized for analytical queries (OLAP).
- **Data Lakehouse:** A hybrid approach using open formats like **Apache Iceberg, Delta Lake, or Apache Hudi**. It brings ACID transactions and schema enforcement to the cheap storage of a Data Lake.

Data Storage and Retrieval Strategies for Data Pipelines

Storage Formats: How it's Written

Choosing the right file format is the easiest way to slash your cloud bill and boost performance.

- **Row-Based (CSV, JSON, Avro):** Good for "Point Lookups" (retrieving a single record). Avro is excellent for streaming pipelines because it handles schema evolution gracefully.
- **Columnar-Based (Parquet, ORC):** The gold standard for analytics. Since most queries only touch a few columns (e.g., "Calculate average price"), columnar formats allow the engine to skip reading the data it doesn't need.

Data Retrieval Strategies in Data Pipelines

Full Data Retrieval Strategy

Retrieve the entire dataset during each pipeline run.

Used when

- Data volume is small
- Data changes frequently

Limitations

- High I/O cost
- Poor scalability

Data Retrieval Strategies in Data Pipelines

Incremental Retrieval Strategy

Retrieve only new or modified data since the last run.

Techniques

- Timestamps
- Change flags
- Version numbers

Benefits

- Faster pipelines
- Lower compute and storage cost

Data Retrieval Strategies in Data Pipelines

Partition-Based Retrieval Strategy

Retrieve only specific partitions of data.

Benefits

- Reduced data scanning
- Improved query performance

Data Retrieval Strategies in Data Pipelines

Index-Based Retrieval Strategy

Use indexes to locate required records efficiently.

Benefits

- Fast point and range queries
- Reduced query latency

Data Retrieval Strategies in Data Pipelines

Predicate Pushdown Strategy

Apply filters at the storage layer before data is loaded.

Benefits

- Minimized data movement
- Faster execution

Data Retrieval Strategies in Data Pipelines

Column Pruning Strategy

Retrieve only the required columns from storage.

Benefits

- Reduced I/O
- Efficient analytical queries

Data Retrieval Strategies in Data Pipelines

Parallel Retrieval Strategy

Retrieve data simultaneously from multiple nodes or partitions.

Benefits

- Higher throughput
- Reduced execution time

Data Retrieval Strategies in Data Pipelines

Caching-Based Retrieval Strategy

Serve frequently accessed data from cache instead of storage.

Benefits

- Low latency access
- Reduced storage load

Data Retrieval Strategies in Data Pipelines

API-Oriented Retrieval Strategy

Retrieve data through controlled APIs.

Benefits

- Secure access
- Query pattern control
- Scalability

Data Retrieval Strategies in Data Pipelines

Batch Retrieval Strategy

Retrieve data in scheduled, large batches.

Used when

- Latency is not critical
- Large historical datasets

Data Retrieval Strategies in Data Pipelines

Streaming Retrieval Strategy

Retrieve data continuously as it arrives.

Used when

- Real-time or near real-time processing is required

Data Retrieval Strategies in Data Pipelines

Access-Controlled Retrieval Strategy

Apply access rules during retrieval.

Benefits

- Data security
- Compliance enforcement

Automated Report Generation (Excel, HTML, PDF)

What is Automated Report Generation?

Automated report generation is the process of **automatically creating reports from data pipelines** at scheduled intervals or events **without manual intervention**.

Reports may include:

- Tables
- Metrics
- Charts
- Summaries
- Insights

Output formats:

- **Excel** → business users
- **HTML** → dashboards / web viewing
- **PDF** → official, shareable documents

Automated Report Generation (Excel, HTML, PDF)

Feature	BI Tool (Looker/Tableau)	Python (Jinja/Plotly)	Notebooks (Papermill)
Effort	Low (Drag & Drop)	High (Coding required)	Medium
Customization	Limited to UI	Infinite	High
Cost	High (Licensing)	Low (Open Source)	Low
Best Use Case	Internal Ops	External Client Reports	Data Science/Exploratory

Case Study

A growing online education platform wants to:

Track student activity, test performance, and revenue

Generate daily and monthly analytics reports automatically

Deliver reports in Excel (operations), HTML (dashboard), and PDF (management) formats

Eliminate manual reporting work